



# Smart Contract Security Audit Report



# Table Of Contents

|                               |       |
|-------------------------------|-------|
| <b>1 Executive Summary</b>    | _____ |
| <b>2 Audit Methodology</b>    | _____ |
| <b>3 Project Overview</b>     | _____ |
| 3.1 Project Introduction      | _____ |
| 3.2 Vulnerability Information | _____ |
| <b>4 Code Overview</b>        | _____ |
| 4.1 Contracts Description     | _____ |
| 4.2 Visibility Description    | _____ |
| 4.3 Vulnerability Summary     | _____ |
| <b>5 Audit Result</b>         | _____ |
| <b>6 Statement</b>            | _____ |

# 1 Executive Summary

On 2024.11.27, the SlowMist security team received the team's security audit application for RingLaunchpad, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

| Test method       | Description                                                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Black box testing | Conduct security tests from an attacker's perspective externally.                                                                     |
| Grey box testing  | Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.        |
| White box testing | Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc. |

The vulnerability severity level information:

| Level      | Description                                                                                                                                                                                                        |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Critical   | Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.                                          |
| High       | High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.                                                                   |
| Medium     | Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.                                                                                 |
| Low        | Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed. |
| Weakness   | There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.                                                                                                                   |
| Suggestion | There are better practices for coding or architecture.                                                                                                                                                             |

## 2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

| Serial Number | Audit Class                    | Audit Subclass                        |
|---------------|--------------------------------|---------------------------------------|
| 1             | Overflow Audit                 | -                                     |
| 2             | Reentrancy Attack Audit        | -                                     |
| 3             | Replay Attack Audit            | -                                     |
| 4             | Flashloan Attack Audit         | -                                     |
| 5             | Race Conditions Audit          | Reordering Attack Audit               |
| 6             | Permission Vulnerability Audit | Access Control Audit                  |
|               |                                | Excessive Authority Audit             |
| 7             | Security Design Audit          | External Module Safe Use Audit        |
|               |                                | Compiler Version Security Audit       |
|               |                                | Hard-coded Address Security Audit     |
|               |                                | Fallback Function Safe Use Audit      |
|               |                                | Show Coding Security Audit            |
|               |                                | Function Return Value Security Audit  |
|               |                                | External Call Function Security Audit |

| Serial Number | Audit Class                           | Audit Subclass                          |
|---------------|---------------------------------------|-----------------------------------------|
| 7             | Security Design Audit                 | Block data Dependence Security Audit    |
|               |                                       | tx.origin Authentication Security Audit |
| 8             | Denial of Service Audit               | -                                       |
| 9             | Gas Optimization Audit                | -                                       |
| 10            | Design Logic Audit                    | -                                       |
| 11            | Variable Coverage Vulnerability Audit | -                                       |
| 12            | "False Top-up" Vulnerability Audit    | -                                       |
| 13            | Scoping and Declarations Audit        | -                                       |
| 14            | Malicious Event Log Audit             | -                                       |
| 15            | Arithmetic Accuracy Deviation Audit   | -                                       |
| 16            | Uninitialized Storage Pointer Audit   | -                                       |

## 3 Project Overview

### 3.1 Project Introduction

This is the RingLaunchpad contract of Ring Protocol, which helps users create and launch new tokens on decentralized exchanges and provide initial liquidity, enabling users to easily deploy tokens, add liquidity, purchase tokens, and set liquidity thresholds, thereby simplifying the entire process of token issuance and liquidity management.

### 3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

| NO | Title                                 | Category                      | Level       | Status       |
|----|---------------------------------------|-------------------------------|-------------|--------------|
| N1 | Malicious creation of liquidity       | Design Logic Audit            | Medium      | Acknowledged |
| N2 | Receive can lock users' native tokens | Others                        | Low         | Acknowledged |
| N3 | Missing zero address check            | Others                        | Suggestion  | Acknowledged |
| N4 | Missing the event records             | Others                        | Suggestion  | Fixed        |
| N5 | Redundant code                        | Others                        | Suggestion  | Fixed        |
| N6 | Predictable Token Deployment Address  | Race Conditions Vulnerability | Information | Acknowledged |

## 4 Code Overview

### 4.1 Contracts Description

#### Audit Version:

<https://github.com/Few-Protocol/ring-launchpad>

commit: f95816434c0910bdf568016f70ecc73a70174a0d

#### Fixed Version:

<https://github.com/Few-Protocol/ring-launchpad>

commit: d75841034d76660456b57e9bd4845527f9cc26d1

The main network address of the contract is as follows:

**The code was not deployed to the mainnet.**

### 4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

| RingLaunchpad   |            |                  |                        |
|-----------------|------------|------------------|------------------------|
| Function Name   | Visibility | Mutability       | Modifiers              |
| <Constructor>   | Public     | Can Modify State | Ownable                |
| deploy          | Public     | Can Modify State | nonReentrant           |
| deployWETH      | External   | Can Modify State | -                      |
| deployETH       | External   | Payable          | -                      |
| _buyTokens      | Private    | Can Modify State | nonReentrant           |
| setThreshold    | External   | Can Modify State | onlyOwner nonReentrant |
| <Receive Ether> | External   | Payable          | -                      |

| Token         |            |                  |           |
|---------------|------------|------------------|-----------|
| Function Name | Visibility | Mutability       | Modifiers |
| <Constructor> | Public     | Can Modify State | ERC20     |

## 4.3 Vulnerability Summary

### [N1] [Medium] Malicious creation of liquidity

#### Category: Design Logic Audit

#### Content

In the RingLaunchpad contract, users can create a new ERC20 Token contract through the publicly callable function `deploy`, and create a new `wrappedToken` contract for this new ERC20 Token contract. And use this `wrappedToken` contract and its `wrappedWETH` as a pair to add a token pair in UniswapV2. Because it is publicly callable and there is no other cost except the gas fee, malicious users can call the `deploy` function multiple times and cast `wrappedWETH` to add token pairs. The `deployWETH` and `deployETH` functions can also be deployed by passing `buyAmount` or `msg.value` as 0 to the `deploy` function. Although there is an upper limit on the `pairAmount`, `wrappedWETH` can be over-minted through multiple calls. Also, the minting Token contract does not check the name and symbol restrictions. Then this malicious user can cast multiple token pairs with the same name and symbol but

different addresses to confuse the user market.

Code location:

contracts/RingLaunchpad.sol#L48-L84, L93, L107

```
function deploy(
    string memory name,
    string memory symbol,
    uint256 pairAmount
) public virtual nonReentrant returns (address token, address wrappedToken) {
    if (
        ...
    )

    IFewWrappedToken(wrappedWETH).mint(address(this), pairAmount);

    token = address(new Token(name, symbol));
    wrappedToken = IFewFactory(wrappingFactory).createToken(token);
    ...
    IUniswapV2Router02(uniswapRouter).addLiquidity(
        ...
    );
    emit TokenDeployed(msg.sender, token, wrappedToken, pairAmount);
}

function deployWETH(
    ...
) external virtual returns (address token, address wrappedToken) {
    (token, wrappedToken) = deploy(name, symbol, pairAmount);

    if (buyAmount > 0) {
        IERC20(weth).safeTransferFrom(msg.sender, address(this), buyAmount);
        _buyTokens(recipient, wrappedToken, buyAmount);
    }
}

function deployETH(
    ...
) external payable virtual returns (address token, address wrappedToken) {
    (token, wrappedToken) = deploy(name, symbol, pairAmount);

    if (msg.value > 0) {
        IWETH(weth).deposit{value: msg.value}();
        _buyTokens(recipient, wrappedToken, msg.value);
    }
}
```



### Solution

It is recommended to authenticate the deploy function, and call the deploy function to create and add token pairs only when a swap call is needed in the deployWETH and deployETH functions. At the same time, add restrictions that limit the name and symbol to be different to prevent confusion in the token market.

### Status

Acknowledged; After communicating with the project team, they stated that this is a known feature. Users need to pay gas to create tokens, so it is fine. Duplicate names and symbols will also happen.

## [N2] [Low] Receive can lock users' native tokens

### Category: Others

### Content

There is a receive function in the RingLaunchpad contract so that the contracts can receive native tokens. However, the receive function can lock users' native tokens when users transfer the native token in these contracts by mistake.

Code location:

contracts/RingLaunchpad.sol#L148

```
receive() external payable {}
```

### Solution

It's recommended to add addresses that can send the native tokens in the contract in the require check logic in the receive() function.

### Status

Acknowledged

## [N3] [Suggestion] Missing zero address check

### Category: Others

### Content

When setting the address in parameter constructor initialization, it is not checked whether the set address is 0 address.

Code location:

contracts/RingLaunchpad.sol#L35-L46

```
constructor(  
    address _uniswapRouter,  
    address _weth,  
    address _wrappedWETH,  
    address _wrappingFactory  
) Ownable(msg.sender) {  
    uniswapRouter = _uniswapRouter;  
    weth = _weth;  
    wrappedWETH = _wrappedWETH;  
    wrappingFactory = _wrappingFactory;  
    threshold = 100 ether;  
}
```

#### Solution

It is recommended to add the 0 address check when sensitive parameters are modified.

#### Status

Acknowledged

#### [N4] [Suggestion] Missing the event records

Category: Others

#### Content

In the RingLaunchpad contract, the owner role can modify the threshold through the setThreshold function, but there is no event record.

Code location:

contracts/RingLaunchpad.sol#L143-L146

```
function setThreshold(uint256 _threshold) external onlyOwner nonReentrant {  
    if (_threshold > MAX_THRESHOLD) revert InvalidParameters();  
    threshold = _threshold;  
}
```

#### Solution

It is recommended to record events when sensitive parameters are modified for self-inspection or community review.

**Status**

Fixed

**[N5] [Suggestion] Redundant code****Category: Others****Content**

In the RingLaunchpad contract, the owner role can modify the threshold through the setThreshold function, but there is a nonReentrant check in the function's modifier. The function only needs to modify this parameter and there is no subsequent operation, so the nonReentrant modifier is redundant.

Code location:

contracts/RingLaunchpad.sol#L143-L146

```
function setThreshold(uint256 _threshold) external onlyOwner nonReentrant {  
    if (_threshold > MAX_THRESHOLD) revert InvalidParameters();  
    threshold = _threshold;  
}
```

**Solution**

It is recommended to clarify the business logic implementation, and if it is redundant code, it is recommended to remove it from the contract to save gas.

**Status**

Fixed

**[N6] [Information] Predictable Token Deployment Address****Category: Race Conditions Vulnerability****Content**

In the deploy function, the new Token(name, symbol) creates a token whose contract address is determined based on the creator's address and nonce. This address can be predicted, making it vulnerable to front-running attacks.

Code location:

contracts/RingLaunchpad.sol#L64

```
token = address(new Token(name, symbol));
```

### Solution

It is recommended approach is to leverage the CREATE2 opcode combined with a salt value to deploy contracts.

### Status

Acknowledged; After communicating with the project team, they stated that they will warn users to double check the actually created token&pool address, never pre-calculate the address and use it as a pre-assumption.

## 5 Audit Result

| Audit Number   | Audit Team             | Audit Date              | Audit Result |
|----------------|------------------------|-------------------------|--------------|
| OX002411270001 | SlowMist Security Team | 2024.11.27 - 2024.11.27 | Medium Risk  |

Summary conclusion: The SlowMist security team uses a manual and SlowMist team's analysis tool to audit the project, during the audit work we found 1 medium risk, 1 low risk, 3 suggestions, and 1 information. All the findings were fixed or acknowledged. The code was not deployed to the mainnet.

## 6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



**Official Website**  
[www.slowmist.com](http://www.slowmist.com)



**E-mail**  
[team@slowmist.com](mailto:team@slowmist.com)



**Twitter**  
[@SlowMist\\_Team](https://twitter.com/SlowMist_Team)



**Github**  
<https://github.com/slowmist>